BUSINESS CASE STUDY · SQL

Target Brazil

E-commerce operations analysis, 2016-2018

SQL analysis of 99,441 orders across customers, sellers, products, payments, freight, delivery and reviews, with insights and actionable recommendations.

99,441	96,096	+137%	12.1 days	4.32 → 1.70
orders, Sep 2016 - Oct 2018	unique customers in 4,119 cities, 27 states	order value, Jan-Aug 2018 vs 2017	average delivery; 6.8% arrive late	review score, early vs 8+ days late

Author: Guntuku Hariprasad

Context: Scaler business case study

Tools: Google BigQuery (Standard SQL) · DuckDB · Python (pandas, matplotlib)

Date: 01 November 2026

Repository: https://github.com/dot-hpg/target-brazil-sql-case-study

Executive summary

- Growth is real and volume-driven. Orders in Jan-Aug 2018 were 135% higher than in Jan-Aug 2017 and payment value rose 137%, while the average order value barely moved (R$159.7 to R$161.0). Target Brazil won more customers; it did not sell them bigger baskets.
- Growth has plateaued. Monthly orders climbed from 800 (Jan 2017) to a Black Friday peak of 7,544 (Nov 2017; 1,176 orders on Black Friday, 24 Nov, alone) and then held at roughly 6,200-7,300 a month through 2018. Year-on-year value growth fell from +705% in January to +52% in August.
- Demand is concentrated in the South-East. SP, RJ and MG hold 66.5% of customers and SP alone 41.9%. 59.7% of sellers are also in SP, so the network is effectively a single hub.
- Distance from the hub drives cost, speed and satisfaction. SP customers get orders in 8.3 days and pay R$15.15 freight per item; Roraima (RR) waits 29.0 days and pays R$42.98. Same-state orders arrive in 7.5 days vs 14.6 cross-state, with 43% lower freight.
- Lateness, not slowness, destroys ratings. Orders delivered early average 4.2-4.3 stars; orders 8+ days late average 1.70 and 70% of them get 1 star. Late rates spike after demand peaks (12.4% in Nov 2017, 19.0% in Mar 2018), so the problem is capacity, not geography alone.
- Credit card is the payment backbone (76.9% of orders, average 3.5 installments). Half of all orders are single-payment, but 10-installment plans carry the biggest tickets (R$415 average).
- Retention is the biggest untapped lever: 96.9% of customers ordered only once.

Top recommendations

1. Fix reliability before speed. A late order loses 0.9-2.6 review stars; an early one gains almost nothing. Track on-time % as the headline logistics KPI, predict late orders and notify/compensate proactively. Start with RJ (13.5% late, second-largest market).

2. Recalibrate delivery estimates per state. Estimates are padded by 11 days on average and by up to 20 days in the North. Showing realistic dates at checkout improves conversion; adding buffer in AL and MA (20-24% late) protects reviews.

3. Localise supply: regional sellers and a North/North-East cross-dock. Same-state items arrive 2x faster with 43% lower freight. 60% of sellers sit in SP while BA, PE and RJ have 70-180 customers per seller. Recruit sellers there and pilot a cross-dock in the North-East.

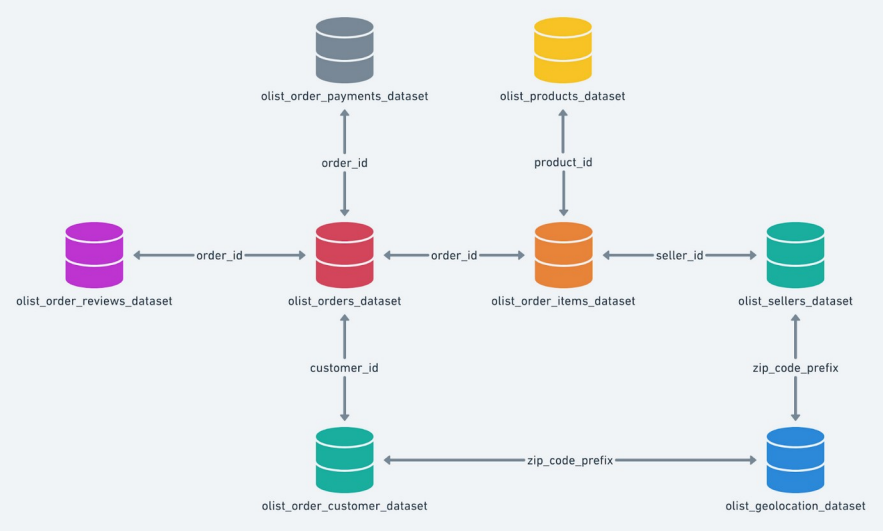
4. Make freight affordable in remote states. Freight is 22-29% of the item price in the North/North-East vs 14% in SP. Bulk carrier contracts, subsidised shipping and free-shipping thresholds would unlock everyday purchases in markets that grew slowest.

All eight recommendations, with the evidence behind each, are in section 8.

Dataset, approach & assumptions

Eight tables (orders, customers, order_items, payments, order_reviews, products, sellers, geolocation) were loaded into BigQuery as dataset target and mirrored locally in DuckDB. Every query below is BigQuery Standard SQL; the identical DuckDB version and the full results are in the repository. Result screenshots show the first 10 rows, as required.

- SQL is written for BigQuery (dataset `target`) and mirrored for DuckDB; both produce identical results.
- Delivery metrics use orders with status 'delivered' and a non-null delivery date (96,470 orders).
- Days are whole 24-hour periods (BigQuery TIMESTAMP_DIFF ... DAY). An order is late when diff_estimated_delivery > 0, i.e. it arrived at least one day after the estimated date.
- 'Average freight value' and 'average delivery time' by state are averaged per item and per order respectively; per-order price and freight are also reported in 4.3-4.4.
- Customer counts use customer_unique_id (one person), not customer_id (one per order).
- Payment type 'UPI' is the label used in this version of the dataset.



Dataset schema: orders is the hub; items link to products and sellers; customers and sellers link to geolocation by zip prefix.

1. Initial exploration: structure & characteristics

Q1.1 Data type of all columns in the customers table

Data type of all columns in the "customers" table.

Query

```
SELECT
  column_name,
  data_type
FROM target.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'customers'
ORDER BY ordinal_position;
```

Result (first 10 rows)

Row	column_name	data_type
1	customer_id	VARCHAR
2	customer_unique_id	VARCHAR
3	customer_zip_code_prefix	BIGINT
4	customer_city	VARCHAR
5	customer_state	VARCHAR

Q1.1 result · 5 rows

Insights

- The customers table has 5 columns. The two IDs, city and state are STRING; the zip code prefix is INT64 (stored as BIGINT locally).
- customer_id is unique per order, while customer_unique_id identifies the real person: 99,441 customer_ids map to 96,096 unique customers. Any customer count in this study uses customer_unique_id.
- Zip prefixes lose their leading zero as integers (01037 becomes 1037). They still join to geolocation correctly because both sides are integers.

Recommendations

- Keep zip prefixes as zero-padded STRING in the warehouse so they match postal formats when shared with logistics partners.

Q1.2 Time range between which the orders were placed

Get the time range between which the orders were placed.

Query

```
SELECT
  MIN(order_purchase_timestamp) AS first_order_at,
  MAX(order_purchase_timestamp) AS last_order_at,
  TIMESTAMP_DIFF(MAX(order_purchase_timestamp), MIN(order_purchase_timestamp), DAY) AS span_in_days,
  COUNT(*) AS total_orders
FROM target.orders;
```

Result (first 10 rows)

Row	first_order_at	last_order_at	span_in_days	total_orders
1	2016-09-04 21:15:19	2018-10-17 17:30:18	772	99,441

Q1.2 result · 1 row

Insights

- Orders span 4 Sep 2016 21:15 to 17 Oct 2018 17:30, about 772 days (25 calendar months).
- The edges are thin: Sep-Dec 2016 has 329 orders (no orders at all in Nov 2016) and Sep-Oct 2018 has 20. Year-on-year comparisons should therefore use Jan-Aug, which the brief also asks for.

Recommendations

- Report trends on complete months (Jan 2017 - Aug 2018) so the partial edges do not distort growth rates.

Q1.3 Count of cities & states of customers who ordered

Count the Cities & States of customers who ordered during the given period.

Query

```
SELECT
  COUNT(DISTINCT c.customer_city)  AS num_cities,
  COUNT(DISTINCT c.customer_state) AS num_states,
  COUNT(DISTINCT c.customer_unique_id) AS num_unique_customers,
  COUNT(DISTINCT o.order_id) AS num_orders
FROM target.orders AS o
JOIN target.customers AS c
  ON o.customer_id = c.customer_id
WHERE o.order_purchase_timestamp BETWEEN
      (SELECT MIN(order_purchase_timestamp) FROM target.orders)
  AND (SELECT MAX(order_purchase_timestamp) FROM target.orders);
```

Result (first 10 rows)

Row	num_cities	num_states	num_unique_customers	num_orders
1	4,119	27	96,096	99,441

Q1.3 result · 1 row

Insights

- Customers who ordered in the period come from 4,119 cities across all 27 Brazilian states (26 states + the Federal District).
- Those 99,441 orders were placed by 96,096 unique customers, i.e. 1.03 orders per customer.

Recommendations

- National reach already exists; the opportunity is deepening penetration outside the South-East, not opening new states.

Q1.4 Data profile: row counts and key coverage for every table

Supporting check: size of every table and how cleanly they join.

Query

```
SELECT 'orders'        AS table_name, COUNT(*) AS row_count, COUNT(DISTINCT order_id)    AS distinct_keys
FROM target.orders
UNION ALL SELECT 'customers',     COUNT(*), COUNT(DISTINCT customer_id)  FROM target.customers
UNION ALL SELECT 'order_items',   COUNT(*), COUNT(DISTINCT order_id)     FROM target.order_items
UNION ALL SELECT 'payments',      COUNT(*), COUNT(DISTINCT order_id)     FROM target.payments
UNION ALL SELECT 'order_reviews', COUNT(*), COUNT(DISTINCT order_id)     FROM target.order_reviews
UNION ALL SELECT 'products',      COUNT(*), COUNT(DISTINCT product_id)   FROM target.products
UNION ALL SELECT 'sellers',       COUNT(*), COUNT(DISTINCT seller_id)    FROM target.sellers
UNION ALL SELECT 'geolocation',   COUNT(*), COUNT(DISTINCT geolocation_zip_code_prefix) FROM
target.geolocation
ORDER BY row_count DESC;
```

Result (first 10 rows)

Row	table_name	row_count	distinct_keys
1	geolocation	1,000,163	19,015
2	order_items	112,650	98,666
3	payments	103,886	99,440
4	customers	99,441	99,441
5	orders	99,441	99,441
6	order_reviews	99,224	98,673
7	products	32,951	32,951
8	sellers	3,095	3,095

Q1.4 result · 8 rows

Insights

- Orders, customers and payments join almost 1:1 (99,441 orders, 99,440 with a payment). 775 orders have no items, mostly cancelled/unavailable ones.
- Geolocation has 1,000,163 rows for only 19,015 zip prefixes: many coordinates per prefix, so it must be aggregated to one row per prefix before joining.
- Reviews hold 99,224 rows for 98,673 orders, so a few orders have more than one review; review scores are averaged per order before joining.

Recommendations

- Publish a deduplicated geolocation view (one centroid per zip prefix) so analysts do not accidentally multiply rows.

Q1.5 Order status mix and missing delivery timestamps

Supporting check: which order statuses exist and how many orders lack delivery dates.

Query

```
SELECT
  order_status,
  COUNT(*) AS orders,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_orders,
  SUM(CASE WHEN order_approved_at IS NULL THEN 1 ELSE 0 END)             AS missing_approved_at,
  SUM(CASE WHEN order_delivered_carrier_date IS NULL THEN 1 ELSE 0 END)  AS missing_carrier_date,
  SUM(CASE WHEN order_delivered_customer_date IS NULL THEN 1 ELSE 0 END) AS missing_customer_date
FROM target.orders
GROUP BY order_status
ORDER BY orders DESC;
```

Result (first 10 rows)

Row	order_status	orders	pct_of_orders	missing_approved_at	missing_carrier_date	missing_customer_date
1	delivered	96,478	97.02	14	2	8
2	shipped	1,107	1.11	0	0	1,107
3	canceled	625	0.63	141	550	619
4	unavailable	609	0.61	0	609	609
5	invoiced	314	0.32	0	314	314
6	processing	301	0.30	0	301	301
7	created	5	0.01	5	5	5
8	approved	2	0.00	0	2	2

Q1.5 result · 8 rows

Insights

- 97.0% of orders were delivered; 1.1% shipped but not delivered, 0.6% cancelled and 0.6% unavailable.
- Only 8 delivered orders lack a delivery date, so delivery metrics cover 96,470 orders.

Recommendations

- Investigate the 609 'unavailable' orders: each one is a sale lost to stock-outs and a likely unhappy customer.

2. In-depth exploration

Q2.1 Is there a growing trend in orders over the years? (yearly view)

Is there a growing trend in the no. of orders placed over the past years?

Query

```
-- 2016 and 2018 are partial years, so the like-for-like comparison is Jan-Aug.
WITH yearly AS (
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8 THEN 1 ELSE 0 END)
      AS jan_aug_orders,
    COUNT(DISTINCT EXTRACT(MONTH FROM order_purchase_timestamp)) AS active_months
  FROM target.orders
  GROUP BY order_year
)
SELECT
  order_year,
  active_months,
  total_orders,
  ROUND(total_orders / active_months, 0) AS avg_orders_per_active_month,
  jan_aug_orders,
  ROUND(100 * (jan_aug_orders - LAG(jan_aug_orders) OVER (ORDER BY order_year))
        / NULLIF(LAG(jan_aug_orders) OVER (ORDER BY order_year), 0), 2) AS jan_aug_yoy_growth_pct
FROM yearly
ORDER BY order_year;
```

Result (first 10 rows)

Row	order_year	active_months	total_orders	avg_orders_per_active_month	jan_aug_orders	jan_aug_yoy_growth_pct
1	2016	3	329	110.00	0	null
2	2017	12	45,101	3,758.00	22,968	null
3	2018	10	54,011	5,401.00	53,991	135.07

Q2.1 result · 3 rows



Insights

- Yes, there is a strong growth trend. Like-for-like (Jan-Aug), orders grew from 22,968 in 2017 to 53,991 in 2018: +135%.
- Average orders per active month rose from 3,758 in 2017 to 5,401 in 2018 (+44%), even though 2018 lacks the Nov-Dec holiday season.

Recommendations

- Use the 2018 run-rate (~6,700 orders a month) as the planning baseline for capacity and inventory, not the 2017 average.

Q2.2 Is there a growing trend in orders? (month-by-month series)

Is there a growing trend in the no. of orders placed over the past years? (monthly series with MoM change and 3-month rolling average)

Query

```
WITH monthly AS (
  SELECT
    FORMAT_TIMESTAMP('%Y-%m', order_purchase_timestamp) AS order_month,
    COUNT(*) AS total_orders
  FROM target.orders
  GROUP BY order_month
)
SELECT
  order_month,
  total_orders,
  total_orders - LAG(total_orders) OVER (ORDER BY order_month) AS mom_change,
  ROUND(100 * (total_orders - LAG(total_orders) OVER (ORDER BY order_month))
        / NULLIF(LAG(total_orders) OVER (ORDER BY order_month), 0), 2) AS mom_change_pct,
  ROUND(AVG(total_orders) OVER (ORDER BY order_month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 0) AS rolling_3m_avg
FROM monthly
ORDER BY order_month;
```

Result (first 10 rows)

Row	order_month	total_orders	mom_change	mom_change_pct	rolling_3m_avg
1	2016-09	4	null	null	4
2	2016-10	324	320.00	8,000.00	164
3	2016-12	1	-323.00	-99.69	110
4	2017-01	800	799.00	79,900.00	375
5	2017-02	1,780	980.00	122.50	860
6	2017-03	2,682	902.00	50.67	1,754
7	2017-04	2,404	-278.00	-10.37	2,289
8	2017-05	3,700	1,296.00	53.91	2,929
9	2017-06	3,245	-455.00	-12.30	3,116
10	2017-07	4,026	781.00	24.07	3,657

Q2.2 result · Showing 10 of 25 rows

Insights

- Orders rose almost every month in 2017, from 800 in January to 4,631 in October.
- November 2017 jumped 62.9% month-on-month to 7,544 orders, the all-time peak; 24 Nov 2017 (Black Friday) alone had 1,176 orders, over 5x the average day that month.
- From January 2018 volume plateaued between 6,167 and 7,269 a month: the growth phase is over and the business is now in a scale phase.

Recommendations

- Re-ignite growth with acquisition campaigns in under-penetrated states, because organic momentum has flattened since Jan 2018.
- Treat Black Friday as the single largest operational event: pre-book carrier capacity and staff from October.

Q2.3 Monthly seasonality in the number of orders

Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

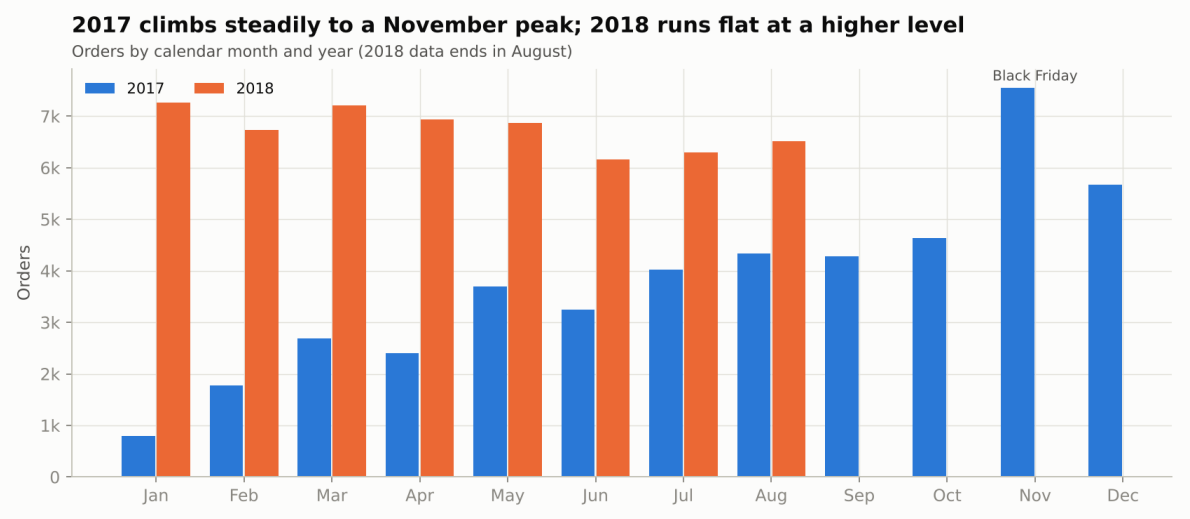
Query

```
-- Calendar month on rows, year on columns, so the same month can be compared across years.
SELECT
  EXTRACT(MONTH FROM order_purchase_timestamp) AS month_no,
  SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2016 THEN 1 ELSE 0 END) AS orders_2016,
  SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2017 THEN 1 ELSE 0 END) AS orders_2017,
  SUM(CASE WHEN EXTRACT(YEAR FROM order_purchase_timestamp) = 2018 THEN 1 ELSE 0 END) AS orders_2018,
  COUNT(*) AS total_orders,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_all_orders
FROM target.orders
GROUP BY month_no
ORDER BY month_no;
```

Result (first 10 rows)

Row	month_no	orders_2016	orders_2017	orders_2018	total_orders	pct_of_all_orders
1	1	0	800	7,269	8,069	8.11
2	2	0	1,780	6,728	8,508	8.56
3	3	0	2,682	7,211	9,893	9.95
4	4	0	2,404	6,939	9,343	9.40
5	5	0	3,700	6,873	10,573	10.63
6	6	0	3,245	6,167	9,412	9.46
7	7	0	4,026	6,292	10,318	10.38
8	8	0	4,331	6,512	10,843	10.90
9	9	4	4,285	16	4,305	4.33
10	10	324	4,631	4	4,959	4.99

Q2.3 result · Showing 10 of 12 rows



Insights

- Seasonality exists but is masked by growth. In 2017 orders rose steadily to a November peak (Black Friday) and eased in December.
- In 2018, Jan-Mar were the strongest months (7,269, 6,728, 7,211) and Jun-Jul the weakest (6,167, 6,292): post-holiday and new-year demand are strong, mid-year is soft.
- May, July and August have the highest share of all orders (10.6-10.9%) because they are covered by two full years of data.

Recommendations

- Run mid-year (June-July) promotions to fill the seasonal dip.
- Build inventory ahead of November and Q1, the two demand peaks.

Q2.4 Time of day when Brazilian customers place orders

During what time of the day do Brazilian customers mostly place their orders? (0-6 Dawn, 7-12 Morning, 13-18 Afternoon, 19-23 Night)

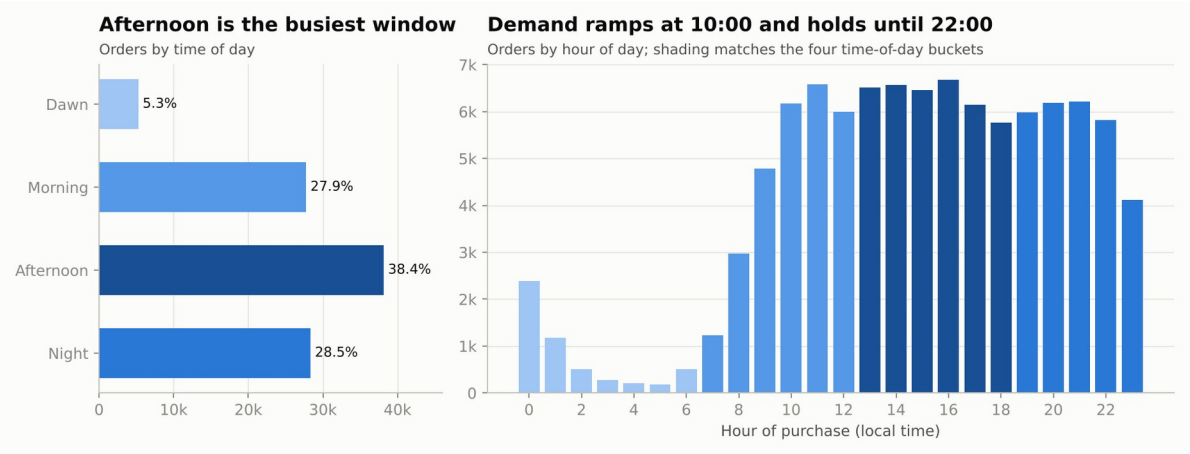
Query

```
SELECT
  CASE
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0  AND 6  THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7  AND 12 THEN 'Morning'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
    ELSE 'Night'
  END AS time_of_day,
  COUNT(*) AS total_orders,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_orders
FROM target.orders
GROUP BY time_of_day
ORDER BY total_orders DESC;
```

Result (first 10 rows)

Row	time_of_day	total_orders	pct_of_orders
1	Afternoon	38,135	38.35
2	Night	28,331	28.49
3	Morning	27,733	27.89
4	Dawn	5,242	5.27

Q2.4 result · 4 rows



Insights

- Brazilian customers order mostly in the Afternoon (38.4%), then Night (28.5%) and Morning (27.9%). Dawn is quiet (5.3%).
- By hour, demand ramps from 10:00 and stays high until 22:00; the busiest hour is 16:00 (6,675 orders).

Recommendations

- Schedule flash sales, push notifications and paid-media bids for 13:00-22:00.
- Use 0:00-6:00 for site deployments and maintenance, when only 5% of orders arrive.

3. Evolution of e-commerce orders in the Brazil region

Q3.1 Month-on-month number of orders placed in each state

Get the month on month no. of orders placed in each state.

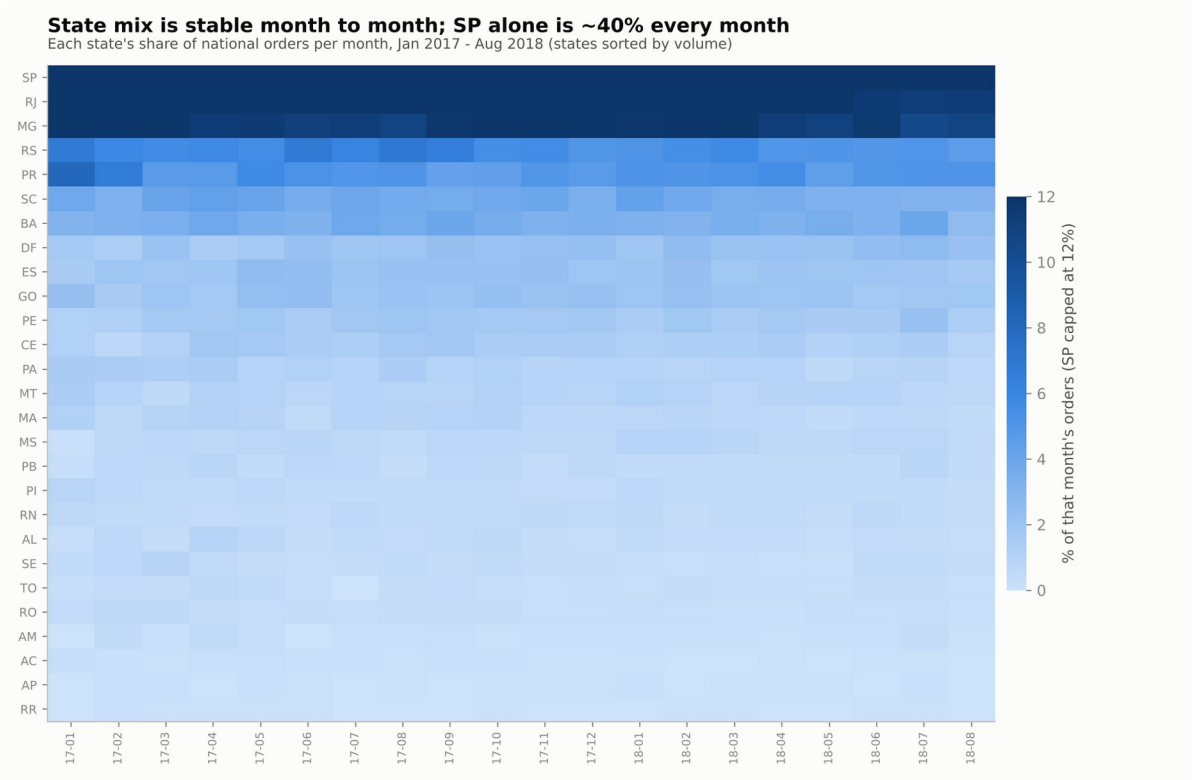
Query

```
WITH state_month AS (
  SELECT
    c.customer_state,
    EXTRACT(YEAR  FROM o.order_purchase_timestamp) AS order_year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp) AS order_month,
    COUNT(o.order_id) AS total_orders
  FROM target.orders AS o
  JOIN target.customers AS c
    ON o.customer_id = c.customer_id
  GROUP BY c.customer_state, order_year, order_month
)
SELECT
  customer_state,
  order_year,
  order_month,
  total_orders,
  total_orders - LAG(total_orders) OVER (
      PARTITION BY customer_state ORDER BY order_year, order_month) AS mom_change
FROM state_month
ORDER BY customer_state, order_year, order_month;
```

Result (first 10 rows)

Row	customer_state	order_year	order_month	total_orders	mom_change
1	AC	2017	1	2	null
2	AC	2017	2	3	1.00
3	AC	2017	3	2	-1.00
4	AC	2017	4	5	3.00
5	AC	2017	5	8	3.00
6	AC	2017	6	4	-4.00
7	AC	2017	7	5	1.00
8	AC	2017	8	4	-1.00
9	AC	2017	9	5	1.00
10	AC	2017	10	6	1.00

Q3.1 result · Showing 10 of 565 rows



Insights

- Every state grew between Jan-Aug 2017 and Jan-Aug 2018 except Acre (AC, 33 to 27 orders).
- The fastest growers were DF (+184%), MS (+172%) and SP (+166%). The North and North-East grew slowest: SE (+43%), RO (+35%), AM (+55%), PA (+66%).
- Each state's share of monthly orders is very stable: SP is ~40% every month.

Recommendations

- Investigate why the North and North-East lag the national growth rate; delivery time and freight (sections 4-5) are the likely blockers.

Q3.2 How customers are distributed across states

How are the customers distributed across all the states?

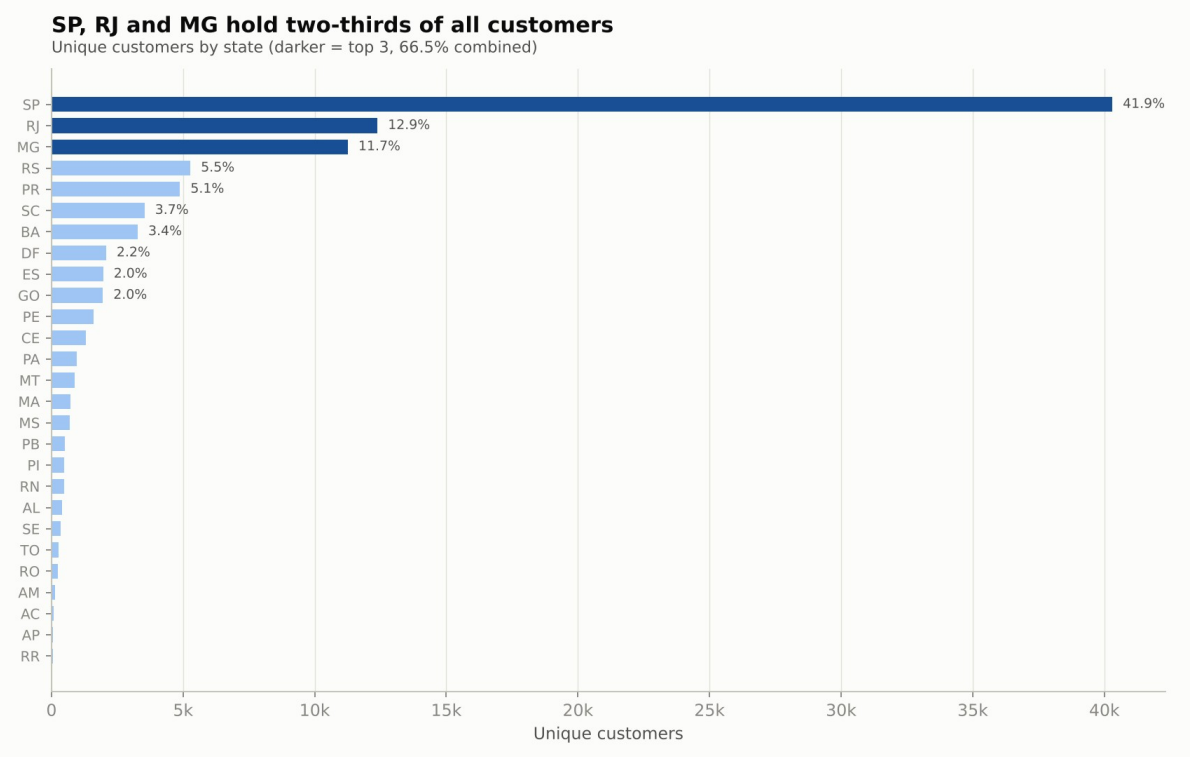
Query

```
WITH by_state AS (
  SELECT
    customer_state,
    COUNT(DISTINCT customer_unique_id) AS unique_customers,
    COUNT(DISTINCT customer_city)      AS cities_covered
  FROM target.customers
  GROUP BY customer_state
)
SELECT
  customer_state,
  unique_customers,
  ROUND(100 * unique_customers / SUM(unique_customers) OVER (), 2) AS pct_of_customers,
  ROUND(100 * SUM(unique_customers) OVER (ORDER BY unique_customers DESC, customer_state
                                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
        / SUM(unique_customers) OVER (), 2) AS cumulative_pct,
  cities_covered
FROM by_state
ORDER BY unique_customers DESC;
```

Result (first 10 rows)

Row	customer_state	unique_customers	pct_of_customers	cumulative_pct	cities_covered
1	SP	40,302	41.92	41.92	629
2	RJ	12,384	12.88	54.80	149
3	MG	11,259	11.71	66.52	745
4	RS	5,277	5.49	72.00	379
5	PR	4,882	5.08	77.08	364
6	SC	3,534	3.68	80.76	240
7	BA	3,277	3.41	84.17	353
8	DF	2,075	2.16	86.33	6
9	ES	1,964	2.04	88.37	95
10	GO	1,952	2.03	90.40	178

Q3.2 result · Showing 10 of 27 rows



Insights

- Customers are highly concentrated: SP has 41.9%, RJ 12.9%, MG 11.7%. The top 5 states (SP, RJ, MG, RS, PR) hold 77.1% of all customers.
- The bottom 5 (RO, AM, AC, AP, RR) together hold only 0.6%. Roraima has just 45 customers.
- SP also covers the most cities in absolute terms after MG (629 vs 745).

Recommendations

- Defend the core (SP, RJ, MG) with loyalty programmes; grow the long tail with regional marketing and cheaper shipping, because each new customer there has low competition.

4. Impact on economy: money movement

Q4.1 % increase in cost of orders, 2017 vs 2018 (Jan-Aug)

Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only), using payment_value.

Query

```
WITH yearly_cost AS (
  SELECT
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
    SUM(p.payment_value) AS total_cost
  FROM target.orders AS o
  JOIN target.payments AS p
    ON o.order_id = p.order_id
  WHERE EXTRACT(YEAR  FROM o.order_purchase_timestamp) IN (2017, 2018)
    AND EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
  GROUP BY order_year
)
SELECT
  ROUND(MAX(CASE WHEN order_year = 2017 THEN total_cost END), 2) AS cost_2017_jan_aug,
  ROUND(MAX(CASE WHEN order_year = 2018 THEN total_cost END), 2) AS cost_2018_jan_aug,
  ROUND(100 * (MAX(CASE WHEN order_year = 2018 THEN total_cost END)
              - MAX(CASE WHEN order_year = 2017 THEN total_cost END))
        / MAX(CASE WHEN order_year = 2017 THEN total_cost END), 2) AS pct_increase
FROM yearly_cost;
```

Result (first 10 rows)

Row	cost_2017_jan_aug	cost_2018_jan_aug	pct_increase
1	3,669,022.12	8,694,733.84	136.98

Q4.1 result · 1 row

Insights

- The cost of orders (payment_value) grew from R$3.67M in Jan-Aug 2017 to R$8.69M in Jan-Aug 2018, an increase of 136.98%.
- Average order value was flat (R$159.7 vs R$161.0), so the increase comes from more orders, not higher prices.

Recommendations

- Add basket-building levers (bundles, free-shipping thresholds, cross-sell) to grow order value now that volume growth has slowed.

Q4.2 Cost of orders by month, 2017 vs 2018 (Jan-Aug)

Supporting view for 4.1: where in the year did the growth come from?

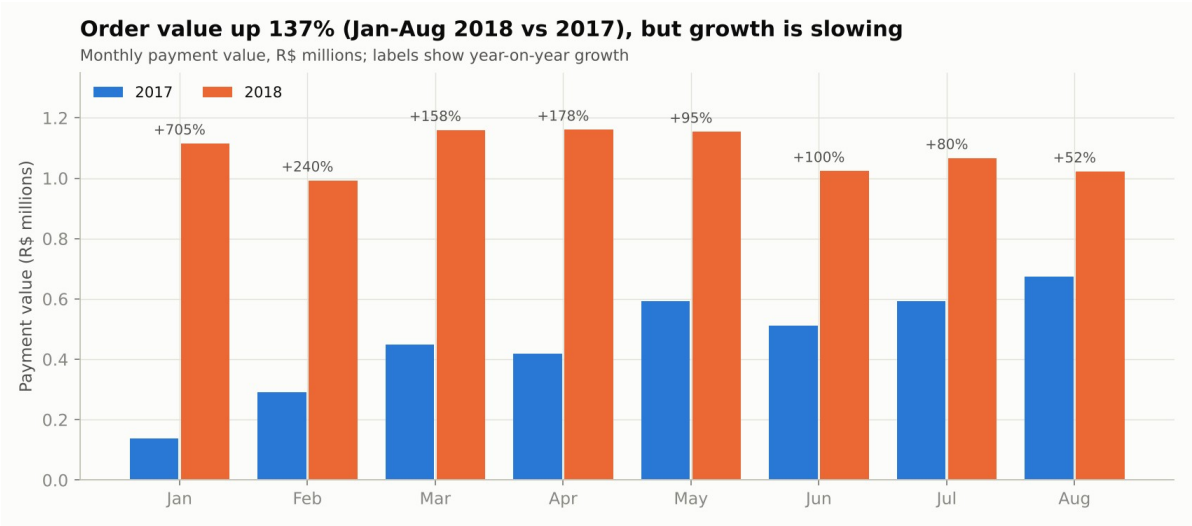
Query

```
SELECT
  EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month_no,
  ROUND(SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 THEN p.payment_value END), 2) AS
cost_2017,
  ROUND(SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018 THEN p.payment_value END), 2) AS
cost_2018,
  ROUND(100 * (SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018 THEN p.payment_value END)
              - SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 THEN p.payment_value END))
        / SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 THEN p.payment_value END), 2) AS
yoy_pct
FROM target.orders AS o
JOIN target.payments AS p
  ON o.order_id = p.order_id
WHERE EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
GROUP BY month_no
ORDER BY month_no;
```

Result (first 10 rows)

Row	month_no	cost_2017	cost_2018	yoy_pct
1	1	138,488.04	1,115,004.18	705.13
2	2	291,908.01	992,463.34	239.99
3	3	449,863.60	1,159,652.12	157.78
4	4	417,788.03	1,160,785.48	177.84
5	5	592,918.82	1,153,982.15	94.63
6	6	511,276.38	1,023,880.50	100.26
7	7	592,382.92	1,066,540.75	80.04
8	8	674,396.32	1,022,425.32	51.61

Q4.2 result · 8 rows



Insights

- Year-on-year growth decays month by month: +705% in January, +178% in April, +52% in August.
- 2018 monthly value is flat at R$1.0-1.16M, confirming the plateau seen in order counts.

Recommendations

- Set 2019 targets on a plateau-aware baseline; extrapolating early-2018 growth rates would overstate demand by a wide margin.

Q4.3 Total & average order price for each state

Calculate the Total & Average value of order price for each state.

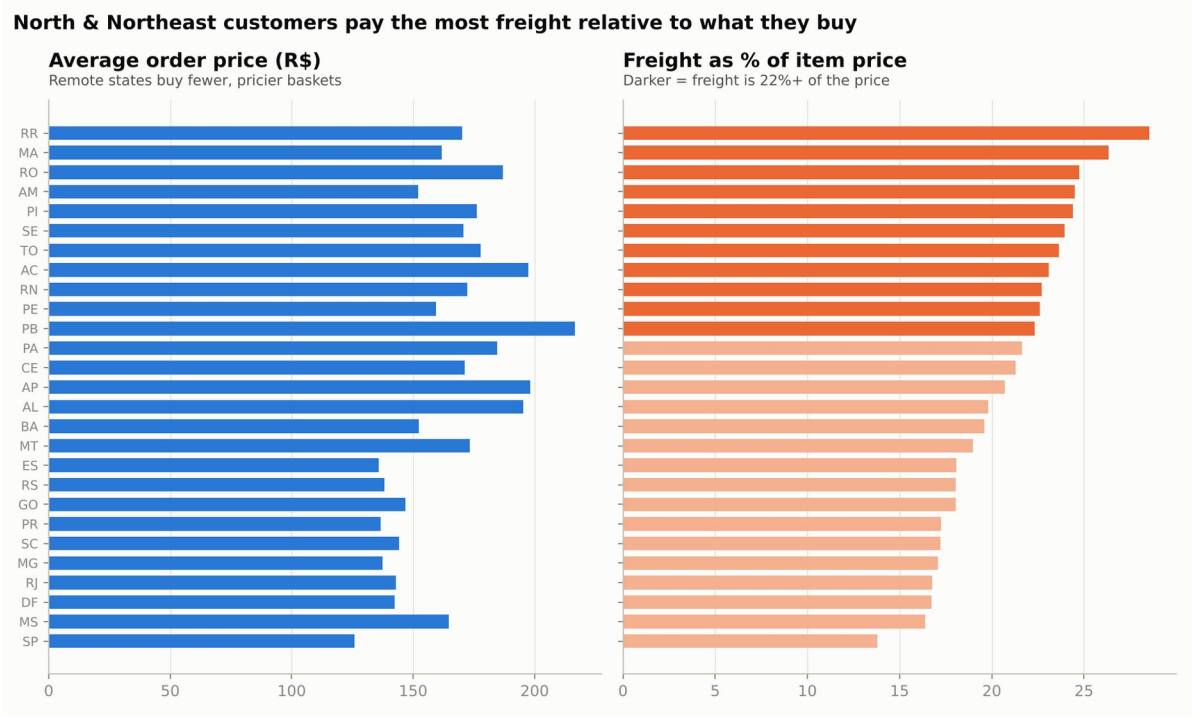
Query

```
SELECT
  c.customer_state,
  ROUND(SUM(oi.price), 2) AS total_order_price,
  ROUND(AVG(oi.price), 2) AS avg_price_per_item,
  ROUND(SUM(oi.price) / COUNT(DISTINCT oi.order_id), 2) AS avg_price_per_order,
  COUNT(DISTINCT oi.order_id) AS orders
FROM target.orders AS o
JOIN target.customers   AS c  ON o.customer_id = c.customer_id
JOIN target.order_items AS oi ON o.order_id = oi.order_id
GROUP BY c.customer_state
ORDER BY total_order_price DESC;
```

Result (first 10 rows)

Row	customer_state	total_order_price	avg_price_per_item	avg_price_per_order	orders
1	SP	5,202,955.05	109.65	125.75	41,375
2	RJ	1,824,092.67	125.12	142.93	12,762
3	MG	1,585,308.03	120.75	137.33	11,544
4	RS	750,304.02	120.34	138.13	5,432
5	PR	683,083.76	119.00	136.67	4,998
6	SC	520,553.34	124.65	144.12	3,612
7	BA	511,349.99	134.60	152.28	3,358
8	DF	302,603.94	125.77	142.40	2,125
9	GO	294,591.95	126.27	146.78	2,007
10	ES	275,037.31	121.91	135.82	2,025

Q4.3 result · Showing 10 of 27 rows



Insights

- SP generates the most revenue by far (R$5.20M, 38% of item value), followed by RJ (R$1.82M) and MG (R$1.59M).
- SP also has the lowest average order price (R$125.75 per order). Remote states buy fewer but pricier baskets: PB R$216.67, AP R$198.15, AC R$197.32 per order.

Recommendations

- In remote states, customers only buy when the item is valuable enough to justify the shipping. Lower freight would unlock everyday, lower-priced purchases there.

Q4.4 Total & average freight value for each state

Calculate the Total & Average value of order freight for each state.

Query

```
SELECT
  c.customer_state,
  ROUND(SUM(oi.freight_value), 2) AS total_freight_value,
  ROUND(AVG(oi.freight_value), 2) AS avg_freight_per_item,
  ROUND(SUM(oi.freight_value) / COUNT(DISTINCT oi.order_id), 2) AS avg_freight_per_order,
  ROUND(100 * SUM(oi.freight_value) / SUM(oi.price), 2) AS freight_as_pct_of_price
FROM target.orders AS o
JOIN target.customers   AS c  ON o.customer_id = c.customer_id
JOIN target.order_items AS oi ON o.order_id = oi.order_id
GROUP BY c.customer_state
ORDER BY total_freight_value DESC;
```

Result (first 10 rows)

Row	customer_state	total_freight_value	avg_freight_per_item	avg_freight_per_order	freight_as_pct_of_price
1	SP	718,723.07	15.15	17.37	13.81
2	RJ	305,589.31	20.96	23.95	16.75
3	MG	270,853.46	20.63	23.46	17.09
4	RS	135,522.74	21.74	24.95	18.06
5	PR	117,851.68	20.53	23.58	17.25
6	BA	100,156.68	26.36	29.83	19.59
7	SC	89,660.26	21.47	24.82	17.22
8	PE	59,449.66	32.92	36.07	22.62
9	GO	53,114.98	22.77	26.46	18.03
10	DF	50,625.50	21.04	23.82	16.73

Q4.4 result · Showing 10 of 27 rows

Insights

- Total freight is largest in SP (R$718.7K) because of volume, but SP's average freight per item is the lowest (R$15.15).
- Freight is a much heavier burden in the North and North-East: 28.6% of the item price in RR, 26.4% in MA and 24.7% in RO, vs 13.8% in SP.

Recommendations

- Introduce regional freight subsidies or free-shipping thresholds for states where freight exceeds 20% of the price.

5. Sales, freight & delivery time

Q5.1 Delivery time and estimated-vs-actual gap per order (single query)

Find the no. of days taken to deliver each order from the purchase date (time_to_deliver) and the difference in days between the actual & estimated delivery date (diff_estimated_delivery), in a single query.

Query

```
-- time_to_deliver         = order_delivered_customer_date - order_purchase_timestamp
-- diff_estimated_delivery = order_delivered_customer_date - order_estimated_delivery_date
--   (negative = delivered BEFORE the estimate, positive = days late;
--    an order counts as late throughout this study when diff_estimated_delivery > 0)
SELECT
  order_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  TIMESTAMP_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)      AS time_to_deliver,
  TIMESTAMP_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS
diff_estimated_delivery
FROM target.orders
WHERE order_status = 'delivered'
  AND order_delivered_customer_date IS NOT NULL
ORDER BY order_purchase_timestamp;
```

Result (first 10 rows)

Row	order_id	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date	time_to_deliver	diff_estimated_delivery
1	bfbd0f9bdef84302105ad712db648a6c	2016-09-15 12:16:38	2016-11-09 07:47:38	2016-10-04	54	36
2	3b697a20d9e427646d92567910af6d57	2016-10-03 09:44:50	2016-10-26 14:02:13	2016-10-27	23	0
3	be5bc2f0da14d8071e2d45451ad119d9	2016-10-03 16:56:50	2016-10-27 18:19:38	2016-11-07	24	-10
4	a41c8759fbe7aab36ea07e038b2d4465	2016-10-03 21:13:36	2016-11-03 10:58:07	2016-11-29	30	-25
5	d207cc272675637bfed0062edffd0818	2016-10-03 22:06:03	2016-10-31 11:07:42	2016-11-23	27	-22
6	cd3b8574c82b42fc8129f6d502690c3e	2016-10-03 22:31:31	2016-10-14 16:08:00	2016-11-23	10	-39
7	ae8a60e4b03c5a4ba9ca0672c164b181	2016-10-03 22:44:10	2016-11-03 14:04:50	2016-12-01	30	-27
8	ef1b29b591d31d57c0d7337460dd83c9	2016-10-03 22:51:30	2016-11-01 15:14:45	2016-11-25	28	-23
9	0a0837a5eee9e7a9ce2b1fa831944d27	2016-10-04 09:06:10	2016-10-22 14:51:18	2016-11-24	18	-32
10	1ff217aa612f6cd7c4255c9bfe931c8b	2016-10-04 09:16:33	2016-10-24 16:33:45	2016-11-24	20	-30

Q5.1 result · Showing 10 of 96,470 rows

Insights

- The query returns one row per delivered order (96,470 rows) with time_to_deliver and diff_estimated_delivery, computed exactly as the brief's formulas.
- A negative diff_estimated_delivery means the order arrived before the estimate; the first rows show typical values of -22 to -39 days in late 2016.

Recommendations

- Store both metrics in a delivery fact table so operations can track them daily.

Q5.2 Delivery performance summary (supporting 5.1)

Supporting view: overall distribution of delivery time and lateness.

Query

```
WITH d AS (
  SELECT
    TIMESTAMP_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)      AS time_to_deliver,
    TIMESTAMP_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS
diff_estimated_delivery
  FROM target.orders
  WHERE order_status = 'delivered'
    AND order_delivered_customer_date IS NOT NULL
)
SELECT
  COUNT(*) AS delivered_orders,
  ROUND(AVG(time_to_deliver), 2) AS avg_days_to_deliver,
  MIN(time_to_deliver) AS min_days,
  MAX(time_to_deliver) AS max_days,
  ROUND(AVG(diff_estimated_delivery), 2) AS avg_days_vs_estimate,
  SUM(CASE WHEN diff_estimated_delivery > 0 THEN 1 ELSE 0 END) AS late_orders,
  ROUND(100 * SUM(CASE WHEN diff_estimated_delivery > 0 THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_late
FROM d;
```

Result (first 10 rows)

Row	delivered_orders	avg_days_to_deliver	min_days	max_days	avg_days_vs_estimate	late_orders	pct_late
1	96,470	12.09	0	209	-10.96	6,534	6.77

Q5.2 result · 1 row

Insights

- Average delivery time is 12.09 days (0 to 209 days).
- On average orders arrive 10.96 days before the estimate, yet 6,534 orders (6.77%) still arrive at least one day late.
- SP (1,820) and RJ (1,495) account for 50.7% of late orders.

Recommendations

- Estimates are padded by ~11 days. Tightening them would make the site more competitive at checkout without changing logistics.

Q5.3 Top 5 states with the highest & lowest average freight value

Find out the top 5 states with the highest & lowest average freight value.

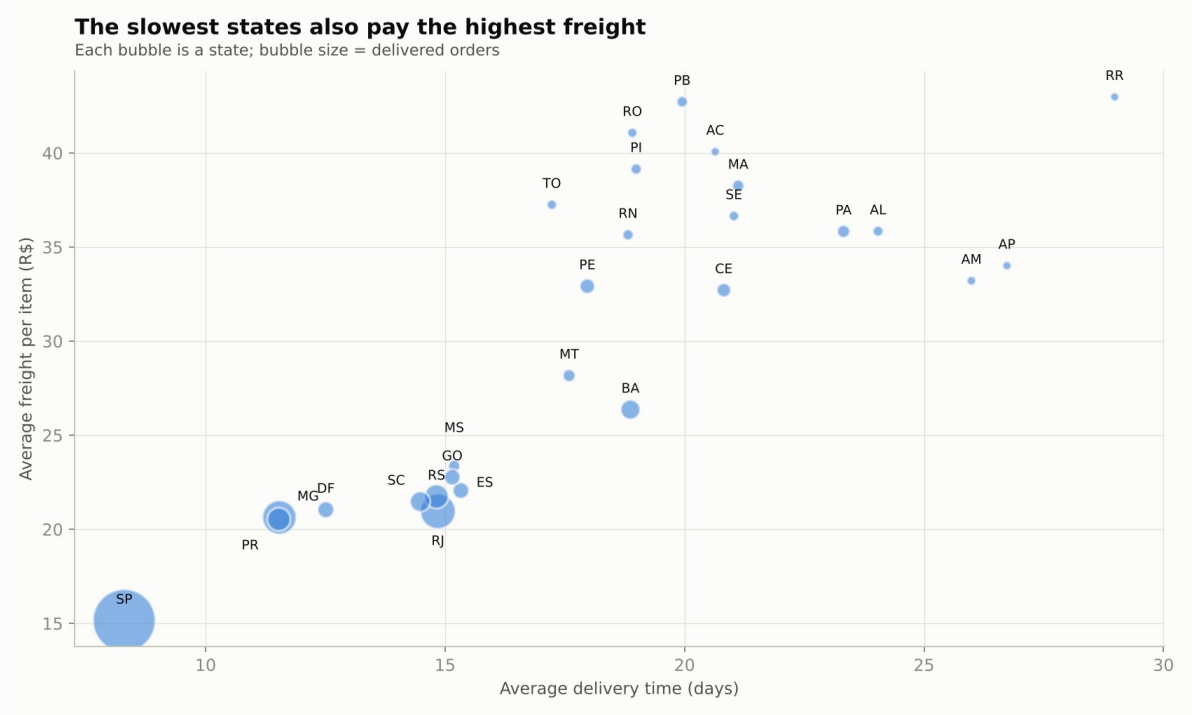
Query

```
WITH state_freight AS (
  SELECT
    c.customer_state,
    ROUND(AVG(oi.freight_value), 2) AS avg_freight_value
  FROM target.orders AS o
  JOIN target.customers   AS c  ON o.customer_id = c.customer_id
  JOIN target.order_items AS oi ON o.order_id = oi.order_id
  GROUP BY c.customer_state
),
ranked AS (
  SELECT
    customer_state,
    avg_freight_value,
    ROW_NUMBER() OVER (ORDER BY avg_freight_value DESC) AS rank_high,
    ROW_NUMBER() OVER (ORDER BY avg_freight_value ASC)  AS rank_low
  FROM state_freight
)
SELECT
  h.rank_high          AS rank_no,
  h.customer_state     AS highest_freight_state,
  h.avg_freight_value  AS highest_avg_freight,
  l.customer_state     AS lowest_freight_state,
  l.avg_freight_value  AS lowest_avg_freight
FROM ranked AS h
JOIN ranked AS l
  ON h.rank_high = l.rank_low
WHERE h.rank_high <= 5
ORDER BY rank_no;
```

Result (first 10 rows)

Row	rank_no	highest_freight_state	highest_avg_freight	lowest_freight_state	lowest_avg_freight
1	1	RR	42.98	SP	15.15
2	2	PB	42.72	PR	20.53
3	3	RO	41.07	MG	20.63
4	4	AC	40.07	RJ	20.96
5	5	PI	39.15	DF	21.04

Q5.3 result · 5 rows



Insights

- Highest average freight: RR (R$42.98), PB (R$42.72), RO (R$41.07), AC (R$40.07), PI (R$39.15), all North/North-East states far from the SP hub.
- Lowest: SP (R$15.15), PR (R$20.53), MG (R$20.63), RJ (R$20.96), DF (R$21.04), the states with the most orders and sellers.
- Freight in RR is 2.8x that in SP.

Recommendations

- Negotiate consolidated (bulk) shipping contracts for the North and North-East, and open a regional cross-dock to shorten the line-haul.

Q5.4 Top 5 states with the highest & lowest average delivery time

Find out the top 5 states with the highest & lowest average delivery time.

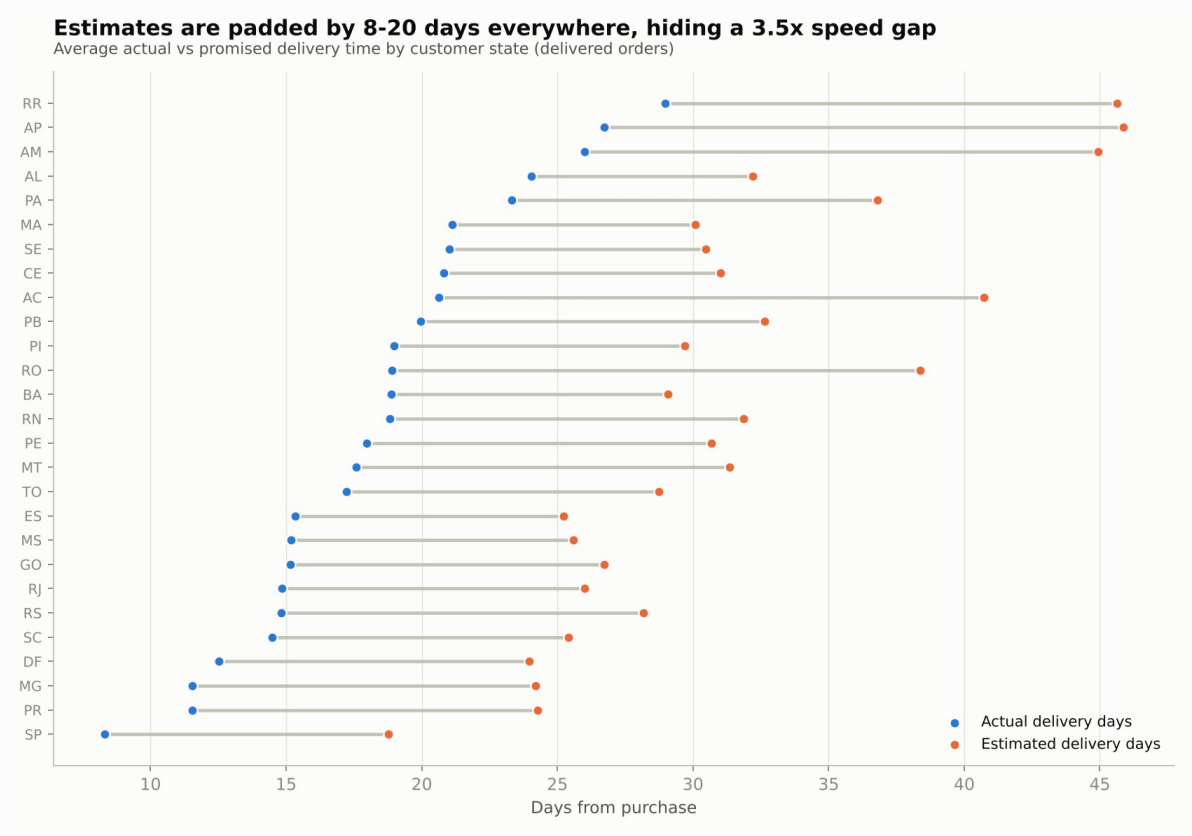
Query

```
WITH state_delivery AS (
  SELECT
    c.customer_state,
    ROUND(AVG(TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY)), 2)
      AS avg_delivery_days
  FROM target.orders AS o
  JOIN target.customers AS c ON o.customer_id = c.customer_id
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
  GROUP BY c.customer_state
),
ranked AS (
  SELECT
    customer_state,
    avg_delivery_days,
    ROW_NUMBER() OVER (ORDER BY avg_delivery_days DESC) AS rank_slow,
    ROW_NUMBER() OVER (ORDER BY avg_delivery_days ASC)  AS rank_fast
  FROM state_delivery
)
SELECT
  s.rank_slow          AS rank_no,
  s.customer_state     AS slowest_state,
  s.avg_delivery_days  AS slowest_avg_days,
  f.customer_state     AS fastest_state,
  f.avg_delivery_days  AS fastest_avg_days
FROM ranked AS s
JOIN ranked AS f
  ON s.rank_slow = f.rank_fast
WHERE s.rank_slow <= 5
ORDER BY rank_no;
```

Result (first 10 rows)

Row	rank_no	slowest_state	slowest_avg_days	fastest_state	fastest_avg_days
1	1	RR	28.98	SP	8.30
2	2	AP	26.73	PR	11.53
3	3	AM	25.99	MG	11.54
4	4	AL	24.04	DF	12.51
5	5	PA	23.32	SC	14.48

Q5.4 result · 5 rows



Insights

- Slowest deliveries: RR (29.0 days), AP (26.7), AM (26.0), AL (24.0), PA (23.3).
- Fastest: SP (8.3 days), PR (11.5), MG (11.5), DF (12.5), SC (14.5).
- A customer in Roraima waits 3.5x longer than one in São Paulo.

Recommendations

- Recruit sellers or place inventory in the North and North-East; same-state orders arrive in half the time (see 7.2).

Q5.5 Top 5 states where delivery is fastest compared to the estimate

Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery (difference between the averages of actual & estimated delivery time).

Query

```
WITH d AS (
  SELECT
    c.customer_state,
    TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY)     AS actual_days,
    TIMESTAMP_DIFF(o.order_estimated_delivery_date, o.order_purchase_timestamp, DAY)     AS estimated_days,
    CASE WHEN TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date, DAY) > 0 THEN
1 ELSE 0 END AS is_late
  FROM target.orders AS o
  JOIN target.customers AS c ON o.customer_id = c.customer_id
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
)
SELECT
  customer_state,
  ROUND(AVG(actual_days), 2)    AS avg_actual_days,
  ROUND(AVG(estimated_days), 2) AS avg_estimated_days,
  ROUND(AVG(estimated_days) - AVG(actual_days), 2) AS days_faster_than_estimate,
  ROUND(100 * AVG(is_late), 2)  AS pct_late
FROM d
GROUP BY customer_state
ORDER BY days_faster_than_estimate DESC
LIMIT 5;
```

Result (first 10 rows)

Row	customer_state	avg_actual_days	avg_estimated_days	days_faster_than_estimate	pct_late
1	AC	20.64	40.73	20.09	3.75
2	RO	18.91	38.39	19.47	2.88
3	AP	26.73	45.87	19.13	2.99
4	AM	25.99	44.92	18.94	2.76
5	RR	28.98	45.63	16.66	12.20

Q5.5 result · 5 rows

Insights

- Deliveries beat the estimate by the widest margin in AC (20.1 days early), RO (19.5), AP (19.1), AM (18.9) and RR (16.7).
- These are the slowest states. They look 'fast' only because their estimates are padded to 38-46 days.
- By contrast, AL (23.9% late) and MA (19.7% late) have the least reliable estimates.

Recommendations

- Recalibrate estimates per state. Over-promising by 20 days loses sales at checkout; under-promising in AL and MA damages reviews.

Q5.6 State delivery scorecard (all 27 states)

Supporting view: delivery speed, buffer vs estimate, lateness and freight side by side for every state.

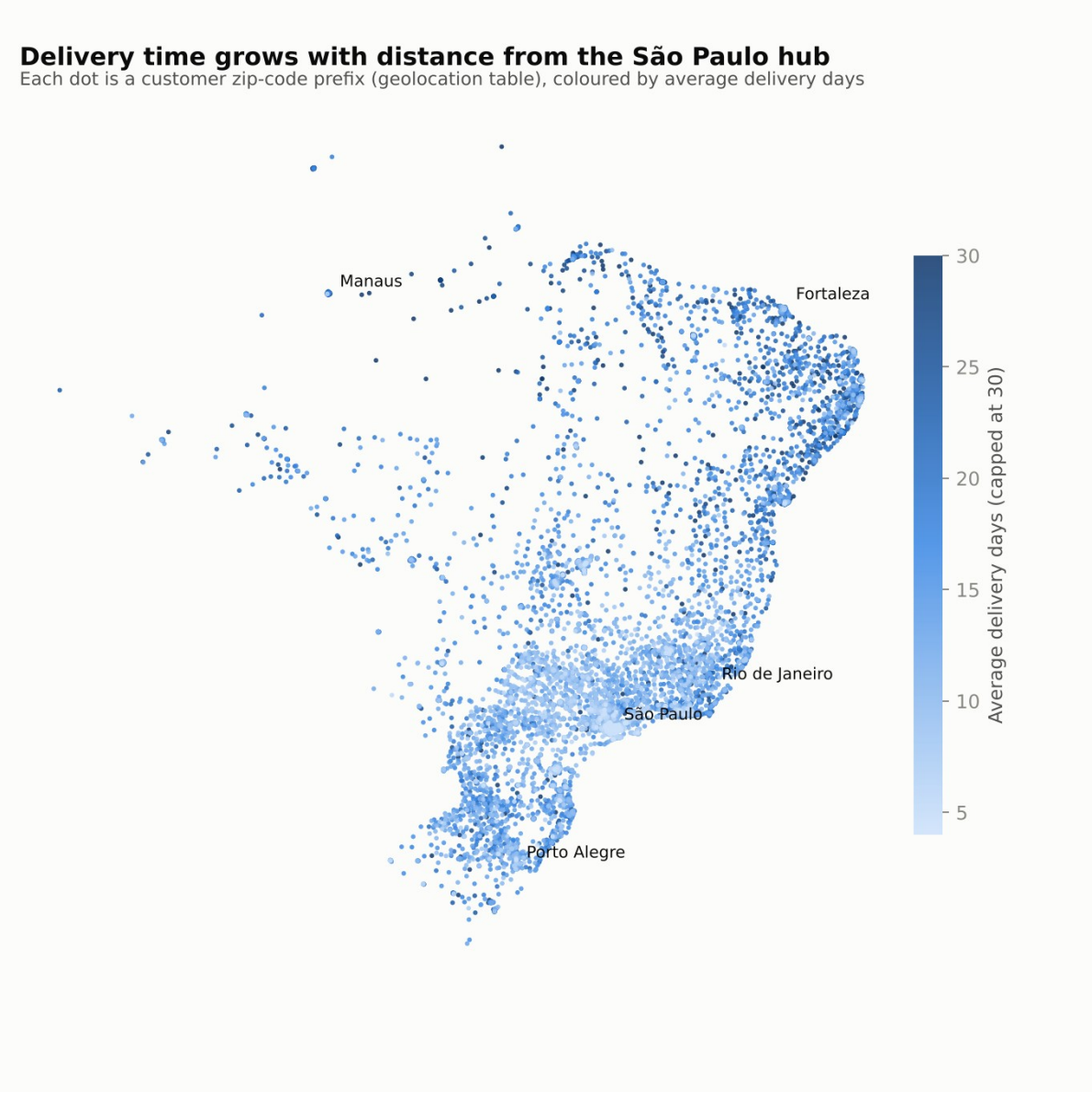
Query

```
WITH d AS (
  SELECT
    c.customer_state,
    o.order_id,
    TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY) AS actual_days,
    TIMESTAMP_DIFF(o.order_estimated_delivery_date, o.order_purchase_timestamp, DAY) AS estimated_days,
    CASE WHEN TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date, DAY) > 0 THEN
1 ELSE 0 END AS is_late
  FROM target.orders AS o
  JOIN target.customers AS c ON o.customer_id = c.customer_id
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
),
f AS (
  SELECT c.customer_state, AVG(oi.freight_value) AS avg_freight
  FROM target.orders AS o
  JOIN target.customers   AS c  ON o.customer_id = c.customer_id
  JOIN target.order_items AS oi ON o.order_id = oi.order_id
  GROUP BY c.customer_state
)
SELECT
  d.customer_state,
  COUNT(*) AS delivered_orders,
  ROUND(AVG(d.actual_days), 2) AS avg_actual_days,
  ROUND(AVG(d.estimated_days) - AVG(d.actual_days), 2) AS days_faster_than_estimate,
  ROUND(100 * AVG(d.is_late), 2) AS pct_late,
  ROUND(MAX(f.avg_freight), 2) AS avg_freight_per_item
FROM d
JOIN f ON d.customer_state = f.customer_state
GROUP BY d.customer_state
ORDER BY avg_actual_days DESC;
```

Result (first 10 rows)

Row	customer_state	delivered_orders	avg_actual_days	days_faster_than_estimate	pct_late	avg_freight_per_item
1	RR	41	28.98	16.66	12.20	42.98
2	AP	67	26.73	19.13	2.99	34.01
3	AM	145	25.99	18.94	2.76	33.21
4	AL	397	24.04	8.17	21.41	35.84
5	PA	946	23.32	13.48	11.21	35.83
6	MA	717	21.12	8.97	17.43	38.26
7	SE	335	21.03	9.45	15.22	36.65
8	CE	1,279	20.82	10.19	13.76	32.71
9	AC	80	20.64	20.09	3.75	40.07
10	PB	517	19.95	12.70	10.44	42.72

Q5.6 result · Showing 10 of 27 rows



Insights

- Delivery time, freight and lateness move together: the North and North-East are slow, expensive and (in AL, MA, CE, SE, PI) often late.
- Among large states, RJ stands out with 13.5% late deliveries despite average speed, 2.3x SP's rate.

Recommendations

- Make RJ the first fix: it is the second-biggest market and has a lateness problem, not a distance problem.

6. Payments

Q6.1 Month-on-month orders by payment type

Find the month on month no. of orders placed using different payment types.

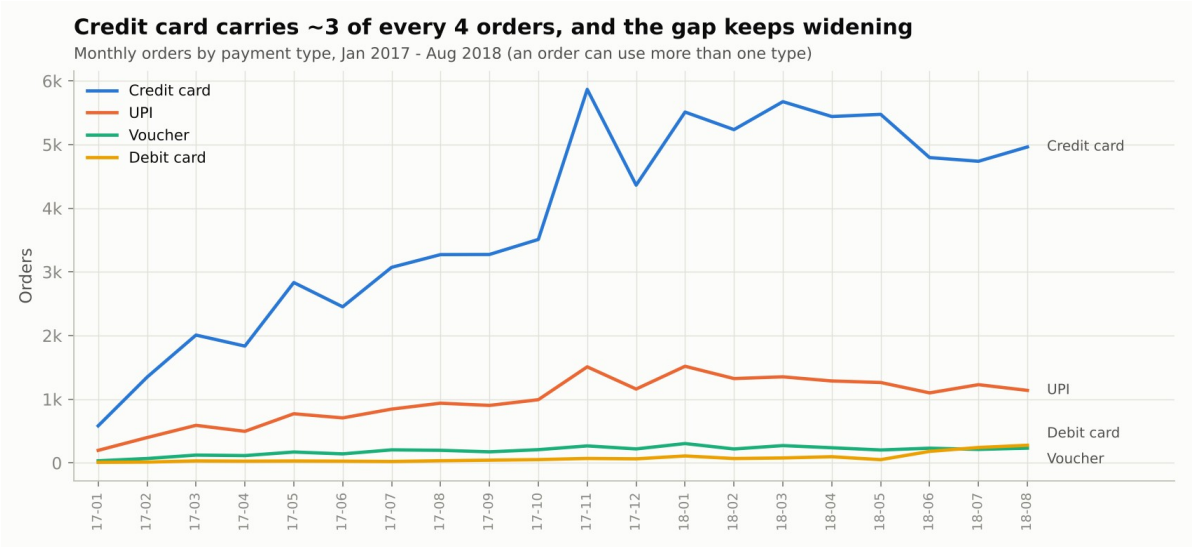
Query

```
SELECT
  FORMAT_TIMESTAMP('%Y-%m', o.order_purchase_timestamp) AS order_month,
  p.payment_type,
  COUNT(DISTINCT o.order_id) AS total_orders
FROM target.orders AS o
JOIN target.payments AS p
  ON o.order_id = p.order_id
GROUP BY order_month, p.payment_type
ORDER BY order_month, total_orders DESC;
```

Result (first 10 rows)

Row	order_month	payment_type	total_orders
1	2016-09	credit_card	3
2	2016-10	credit_card	253
3	2016-10	UPI	63
4	2016-10	voucher	11
5	2016-10	debit_card	2
6	2016-12	credit_card	1
7	2017-01	credit_card	582
8	2017-01	UPI	197
9	2017-01	voucher	33
10	2017-01	debit_card	9

Q6.1 result · Showing 10 of 90 rows



Insights

- Credit card is the dominant method every month and grew fastest, from 582 orders (Jan 2017) to ~5,000 a month in 2018.
- UPI is a steady second at ~1,100-1,500 orders a month. Voucher and debit card are small; debit card picked up in mid-2018.
- All methods peak together in November 2017.

Recommendations

- Partner with card issuers for interest-free installment campaigns around Black Friday, when card volume peaks.

Q6.2 Payment type share, value and installments (supporting 6.1)

Supporting view: share of orders and value by payment type.

Query

```
SELECT
  payment_type,
  COUNT(DISTINCT order_id) AS orders,
  ROUND(100 * COUNT(DISTINCT order_id) / (SELECT COUNT(DISTINCT order_id) FROM target.payments), 2)
    AS pct_of_orders,
  ROUND(SUM(payment_value), 2) AS total_value,
  ROUND(AVG(payment_value), 2) AS avg_payment_value,
  ROUND(AVG(payment_installments), 2) AS avg_installments
FROM target.payments
GROUP BY payment_type
ORDER BY orders DESC;
```

Result (first 10 rows)

Row	payment_type	orders	pct_of_orders	total_value	avg_payment_value	avg_installments
1	credit_card	76,505	76.94	12,542,084.19	163.32	3.51
2	UPI	19,784	19.90	2,869,361.27	145.03	1.00
3	voucher	3,866	3.89	379,436.87	65.70	1.00
4	debit_card	1,528	1.54	217,989.79	142.57	1.00
5	not_defined	3	0.00	0.00	0.00	1.00

Q6.2 result · 5 rows

Insights

- Credit card is used in 76.9% of orders and carries R$12.5M of value (average R$163, 3.5 installments).
- UPI covers 19.9% of orders; vouchers 3.9% with the smallest average ticket (R$65.7).

Recommendations

- Offer a small UPI discount to shift volume from card (lower fees) on low-value baskets.

Q6.3 Number of orders by payment installments

Find the no. of orders placed on the basis of the payment installments that have been paid.

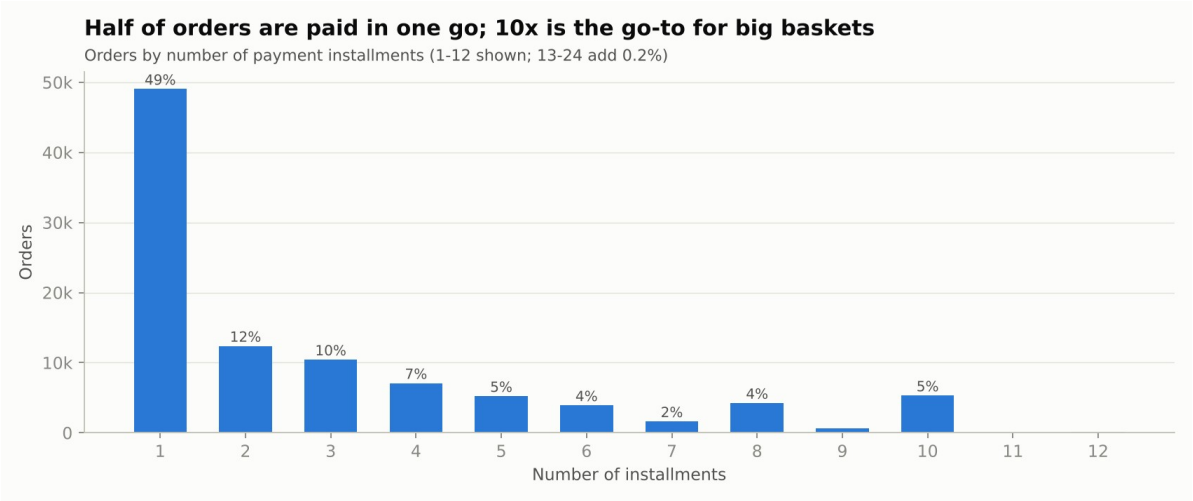
Query

```
SELECT
  payment_installments,
  COUNT(DISTINCT order_id) AS total_orders,
  ROUND(100 * COUNT(DISTINCT order_id)
        / SUM(COUNT(DISTINCT order_id)) OVER (), 2) AS pct_of_orders,
  ROUND(AVG(payment_value), 2) AS avg_payment_value
FROM target.payments
GROUP BY payment_installments
ORDER BY payment_installments;
```

Result (first 10 rows)

Row	payment_installments	total_orders	pct_of_orders	avg_payment_value
1	0	2	0.00	94.32
2	1	49,060	48.91	112.42
3	2	12,389	12.35	127.23
4	3	10,443	10.41	142.54
5	4	7,088	7.07	163.98
6	5	5,234	5.22	183.47
7	6	3,916	3.90	209.85
8	7	1,623	1.62	187.67
9	8	4,253	4.24	307.74
10	9	644	0.64	203.44

Q6.3 result · Showing 10 of 24 rows



Insights

- 48.9% of orders are paid in a single installment, 12.4% in 2 and 10.4% in 3.
- The average payment rises with installments: R$112 for 1, R$308 for 8, R$415 for 10. Installments are what make expensive items affordable.
- 10 installments (5.3% of orders) is a clear spike, the usual 'interest-free 10x' offer in Brazil.

Recommendations

- Promote interest-free installments on high-ticket categories (watches, electronics, furniture) to grow basket size.

7. Bonus deep-dives (beyond the brief)

Q7.1 Does late delivery hurt review scores?

Bonus: average review score and share of 1-star reviews by how early or late the order arrived.

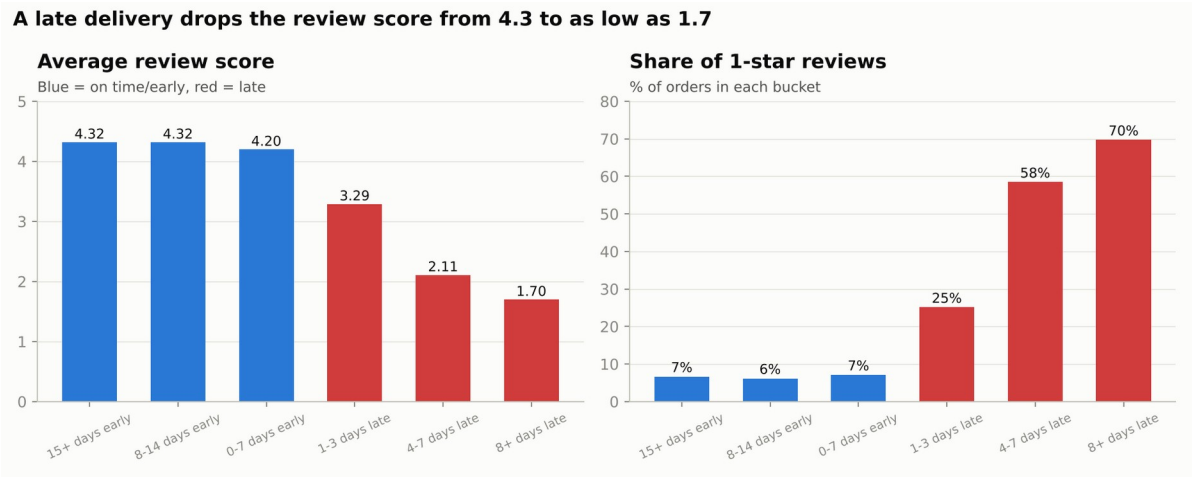
Query

```
WITH d AS (
  SELECT
    o.order_id,
    TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date, DAY) AS days_vs_estimate
  FROM target.orders AS o
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
),
r AS (
  SELECT order_id, AVG(review_score) AS review_score
  FROM target.order_reviews
  GROUP BY order_id
)
SELECT
  CASE
    WHEN d.days_vs_estimate <= -15 THEN '1. 15+ days early'
    WHEN d.days_vs_estimate <= -8  THEN '2. 8-14 days early'
    WHEN d.days_vs_estimate <= 0   THEN '3. 0-7 days early'
    WHEN d.days_vs_estimate <= 3   THEN '4. 1-3 days late'
    WHEN d.days_vs_estimate <= 7   THEN '5. 4-7 days late'
    ELSE                                '6. 8+ days late'
  END AS delivery_bucket,
  COUNT(*) AS orders,
  ROUND(AVG(r.review_score), 2) AS avg_review_score,
  ROUND(100 * AVG(CASE WHEN r.review_score <= 1 THEN 1 ELSE 0 END), 2) AS pct_1_star,
  ROUND(100 * AVG(CASE WHEN r.review_score >= 5 THEN 1 ELSE 0 END), 2) AS pct_5_star
FROM d
JOIN r ON d.order_id = r.order_id
GROUP BY delivery_bucket
ORDER BY delivery_bucket;
```

Result (first 10 rows)

Row	delivery_bucket	orders	avg_review_score	pct_1_star	pct_5_star
1	1. 15+ days early	29,453	4.32	6.66	64.15
2	2. 8-14 days early	36,676	4.32	6.18	63.49
3	3. 0-7 days early	23,314	4.20	7.11	57.75
4	4. 1-3 days late	1,852	3.29	25.11	33.15
5	5. 4-7 days late	1,748	2.11	58.47	14.07
6	6. 8+ days late	2,781	1.70	69.72	7.01

Q7.1 result · 6 rows



Insights

- Customers barely reward early delivery (4.32 stars at 15+ days early vs 4.20 at 0-7 days early), but they punish lateness hard: 3.29 at 1-3 days late, 2.11 at 4-7, 1.70 at 8+.
- 70% of orders delivered 8+ days late receive a 1-star review, vs 6-7% of on-time orders.

Recommendations

- Optimise for reliability (on-time %) rather than raw speed; it is the strongest single driver of ratings in the data.
- Proactively notify and compensate customers (voucher) when an order is predicted to be late.

Q7.2 Same-state vs cross-state fulfilment

Bonus: how much faster and cheaper is an order when the seller is in the customer's state?

Query

```
WITH item_level AS (
  SELECT
    o.order_id,
    CASE WHEN s.seller_state = c.customer_state THEN 'Same state' ELSE 'Cross state' END AS route,
    oi.freight_value,
    oi.price,
    TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY) AS actual_days,
    CASE WHEN TIMESTAMP_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date, DAY) > 0 THEN
1 ELSE 0 END AS is_late
  FROM target.orders AS o
  JOIN target.customers   AS c  ON o.customer_id = c.customer_id
  JOIN target.order_items AS oi ON o.order_id = oi.order_id
  JOIN target.sellers     AS s  ON oi.seller_id = s.seller_id
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_customer_date IS NOT NULL
)
SELECT
  route,
  COUNT(*) AS items,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_items,
  ROUND(AVG(actual_days), 2) AS avg_delivery_days,
  ROUND(AVG(freight_value), 2) AS avg_freight,
  ROUND(100 * SUM(freight_value) / SUM(price), 2) AS freight_pct_of_price,
  ROUND(100 * AVG(is_late), 2) AS pct_late
FROM item_level
GROUP BY route
ORDER BY items DESC;
```

Result (first 10 rows)

Row	route	items	pct_of_items	avg_delivery_days	avg_freight	freight_pct_of_price	pct_late
1	Cross state	70,328	63.82	14.58	23.63	18.29	7.81
2	Same state	39,861	36.18	7.46	13.45	12.97	4.45

Q7.2 result · 2 rows



Insights

- 36.2% of items ship from a seller in the customer's own state.
- Those items arrive in 7.5 days vs 14.6 cross-state, pay R$13.45 vs R$23.63 freight (-43%), and are late 4.5% vs 7.8% of the time.

Recommendations

- Raising the same-state share is the single biggest logistics lever: recruit local sellers and prioritise local stock in search ranking.

Q7.3 Where are the sellers? Seller vs customer concentration

Bonus: share of sellers vs share of customers by state (supply-demand gap).

Query

```
WITH sellers_by_state AS (
  SELECT seller_state AS state, COUNT(*) AS sellers
  FROM target.sellers
  GROUP BY seller_state
),
customers_by_state AS (
  SELECT customer_state AS state, COUNT(DISTINCT customer_unique_id) AS customers
  FROM target.customers
  GROUP BY customer_state
)
SELECT
  c.state,
  c.customers,
  COALESCE(s.sellers, 0) AS sellers,
  ROUND(100 * c.customers / SUM(c.customers) OVER (), 2) AS pct_customers,
  ROUND(100 * COALESCE(s.sellers, 0) / SUM(COALESCE(s.sellers, 0)) OVER (), 2) AS pct_sellers,
  ROUND(c.customers / NULLIF(s.sellers, 0), 1) AS customers_per_seller
FROM customers_by_state AS c
LEFT JOIN sellers_by_state AS s ON c.state = s.state
ORDER BY c.customers DESC;
```

Result (first 10 rows)

Row	state	customers	sellers	pct_customers	pct_sellers	customers_per_seller
1	SP	40,302	1,849	41.92	59.74	21.80
2	RJ	12,384	171	12.88	5.53	72.40
3	MG	11,259	244	11.71	7.88	46.10
4	RS	5,277	129	5.49	4.17	40.90
5	PR	4,882	349	5.08	11.28	14.00
6	SC	3,534	190	3.68	6.14	18.60
7	BA	3,277	19	3.41	0.61	172.50
8	DF	2,075	30	2.16	0.97	69.20
9	ES	1,964	23	2.04	0.74	85.40
10	GO	1,952	40	2.03	1.29	48.80

Q7.3 result · Showing 10 of 27 rows

Insights

- 59.7% of sellers are in SP, which has 41.9% of customers.
- BA (172 customers per seller), PE (179) and RJ (72) are badly under-supplied, compared with 22 in SP.

Recommendations

- Run seller-acquisition drives in BA, PE, CE and RJ to localise supply for large, under-served markets.

Q7.4 Top product categories by revenue

Bonus: which categories drive revenue, and how satisfied are their buyers?

Query

```
WITH r AS (
  SELECT order_id, AVG(review_score) AS review_score
  FROM target.order_reviews
  GROUP BY order_id
)
SELECT
  COALESCE(p.product_category_name, 'unknown') AS category,
  COUNT(DISTINCT oi.order_id) AS orders,
  ROUND(SUM(oi.price), 2) AS revenue,
  ROUND(100 * SUM(oi.price) / SUM(SUM(oi.price)) OVER (), 2) AS pct_revenue,
  ROUND(AVG(oi.price), 2) AS avg_item_price,
  ROUND(AVG(r.review_score), 2) AS avg_review_score
FROM target.order_items AS oi
LEFT JOIN target.products AS p ON oi.product_id = p.product_id
LEFT JOIN r ON oi.order_id = r.order_id
GROUP BY category
ORDER BY revenue DESC
LIMIT 15;
```

Result (first 10 rows)

Row	category	orders	revenue	pct_revenue	avg_item_price	avg_review_score
1	HEALTH BEAUTY	8,836	1,258,681.34	9.26	130.16	4.14
2	Watches present	5,624	1,205,005.68	8.87	201.14	4.02
3	bed table bath	9,417	1,036,988.68	7.63	93.30	3.90
4	sport leisure	7,720	988,048.97	7.27	114.34	4.11
5	computer accessories	6,689	911,954.32	6.71	116.51	3.93
6	Furniture Decoration	6,449	729,762.49	5.37	87.56	3.91
7	Cool Stuff	3,632	635,290.85	4.67	167.36	4.15
8	housewares	5,884	632,248.66	4.65	90.79	4.05
9	automotive	3,897	592,720.11	4.36	139.96	4.06
10	Garden tools	3,518	485,256.46	3.57	111.63	4.05

Q7.4 result · Showing 10 of 15 rows

Insights

- Health & beauty (9.3% of revenue), watches & gifts (8.9%) and bed/table/bath (7.6%) lead revenue.
- Bed/table/bath, furniture/decor and computer accessories have the lowest ratings in the top 15 (3.90-3.93), pointing to quality or expectation gaps.

Recommendations

- Audit sellers and listings in low-rated, high-revenue categories (photos, size guides, packaging).

Q7.5 How many customers come back?

Bonus: distribution of customers by number of orders placed (uses customer_unique_id).

Query

```
WITH per_customer AS (
  SELECT c.customer_unique_id, COUNT(DISTINCT o.order_id) AS orders
  FROM target.orders AS o
  JOIN target.customers AS c ON o.customer_id = c.customer_id
  GROUP BY c.customer_unique_id
)
SELECT
  CASE WHEN orders >= 4 THEN '4+' ELSE CAST(orders AS STRING) END AS orders_per_customer,
  COUNT(*) AS customers,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_customers
FROM per_customer
GROUP BY orders_per_customer
ORDER BY orders_per_customer;
```

Result (first 10 rows)

Row	orders_per_customer	customers	pct_of_customers
1	1	93,099	96.88
2	2	2,745	2.86
3	3	203	0.21
4	4+	49	0.05

Q7.5 result · 4 rows

Insights

- 96.9% of customers ordered only once; 2.9% ordered twice and just 0.26% three or more times.

Recommendations

- Launch a retention programme (post-delivery coupons, loyalty points, personalised recommendations). Even a few points of repeat rate is worth thousands of orders.

Q7.6 Do peaks break the delivery network?

Bonus: monthly share of late deliveries and average delivery time (delivered orders, Jan 2017 - Aug 2018).

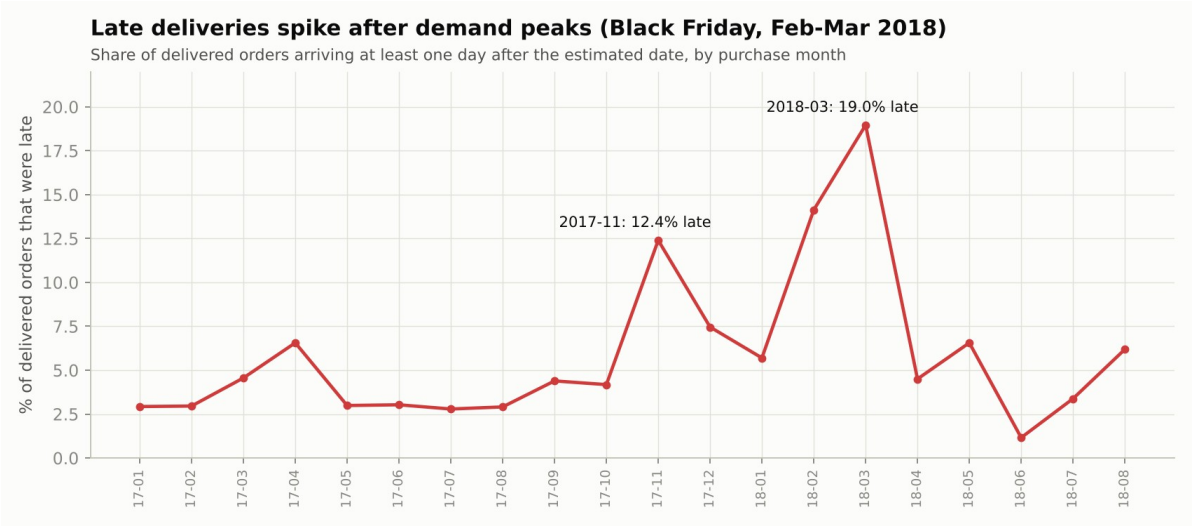
Query

```
SELECT
  FORMAT_TIMESTAMP('%Y-%m', order_purchase_timestamp) AS order_month,
  COUNT(*) AS delivered_orders,
  ROUND(AVG(TIMESTAMP_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)), 2) AS
avg_delivery_days,
  ROUND(100 * AVG(CASE WHEN TIMESTAMP_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY)
> 0
                       THEN 1 ELSE 0 END), 2) AS pct_late
FROM target.orders
WHERE order_status = 'delivered'
  AND order_delivered_customer_date IS NOT NULL
  AND order_purchase_timestamp >= '2017-01-01'
  AND order_purchase_timestamp <  '2018-09-01'
GROUP BY order_month
ORDER BY order_month;
```

Result (first 10 rows)

Row	order_month	delivered_orders	avg_delivery_days	pct_late
1	2017-01	750	12.09	2.93
2	2017-02	1,653	12.61	2.96
3	2017-03	2,546	12.40	4.56
4	2017-04	2,303	14.35	6.56
5	2017-05	3,545	10.76	2.99
6	2017-06	3,135	11.51	3.03
7	2017-07	3,872	11.13	2.79
8	2017-08	4,193	10.70	2.91
9	2017-09	4,150	11.40	4.39
10	2017-10	4,478	11.42	4.18

Q7.6 result · Showing 10 of 20 rows



Insights

- Late deliveries are usually 3-5% of orders but jump after demand peaks: 12.4% in Nov 2017 and 14.1% / 19.0% in Feb / Mar 2018.
- Average delivery time improved to 7.7 days by Aug 2018, so the network can perform when it is not overloaded.

Recommendations

- Plan peak capacity (carriers, warehouse staff) 4-6 weeks ahead of Black Friday and the post-holiday surge, and widen estimates temporarily during peaks.

8. Actionable insights & recommendations

Ranked by expected impact on customer satisfaction and revenue. Each one is backed by the queries referenced above.

1. Fix reliability before speed

A late order loses 0.9-2.6 review stars; an early one gains almost nothing. Track on-time % as the headline logistics KPI, predict late orders and notify/compensate proactively. Start with RJ (13.5% late, second-largest market).

2. Recalibrate delivery estimates per state

Estimates are padded by 11 days on average and by up to 20 days in the North. Showing realistic dates at checkout improves conversion; adding buffer in AL and MA (20-24% late) protects reviews.

3. Localise supply: regional sellers and a North/North-East cross-dock

Same-state items arrive 2x faster with 43% lower freight. 60% of sellers sit in SP while BA, PE and RJ have 70-180 customers per seller. Recruit sellers there and pilot a cross-dock in the North-East.

4. Make freight affordable in remote states

Freight is 22-29% of the item price in the North/North-East vs 14% in SP. Bulk carrier contracts, subsidised shipping and free-shipping thresholds would unlock everyday purchases in markets that grew slowest.

5. Plan peak capacity ahead of Black Friday and Q1

Black Friday 2017 brought 5x a normal day's orders; late deliveries jumped to 12.4% that month and to 14-19% after the Q1 2018 surge. Pre-book carrier capacity and staff from October and temporarily widen estimates during peaks.

6. Grow basket size with installments and bundles

Average order value has been flat at ~R$160 for two years. 10x interest-free plans already carry R$415 tickets; promote them on high-ticket categories and add bundles and free-shipping thresholds.

7. Build retention

97% of customers buy once. A loyalty programme, post-delivery coupons and personalised recommendations target the cheapest growth available, especially now that acquisition-led growth has plateaued.

8. Time marketing to the afternoon and evening

38% of orders arrive 13:00-18:00 and 28% at night. Concentrate flash sales, push and paid-media budget in 13:00-22:00, and run maintenance at dawn.

Suggested KPIs to track

KPI	Current value	Target direction
On-time delivery rate	93.2% (6.8% late)	≥ 96%, and ≤ 8% in every month
Average delivery days	12.1 (SP 8.3, RR 29.0)	≤ 10 nationally; halve North gap
Estimate padding (est. - actual)	11.0 days early on average	3-5 days, per state
Same-state fulfilment share	36.2% of items	≥ 45%
Freight as % of price (North/NE)	20-29%	≤ 18%
Repeat-customer rate	3.1%	≥ 6%
Average review score	4.09	≥ 4.3

Code, data, notebook and full results: https://github.com/dot-hpg/target-brazil-sql-case-study